Agentic Expense Claims: A Multi-Agent Multimodal System for Automated Expense Report Processing

Nguyen Thanh Tung	Josiah Lau	James Oon	Sagar Pratap Singh
1011011	1001530	1010718	1010736

1 Problem Decomposition

1.1 Problem Statement

The expense claim process in many corporate environments including SUTD using SAP Concur contains operational inefficiencies for both students and administrative reviewers. Reimbursements may take up to two months due to multiple validation stages, manual data entry, policy interpretation challenges, and repeated claim resubmissions that often lead to errors and claim rejections.

1.2 Current Workflow Pain Points

The current workflow faces several inefficiencies, including OCR errors in receipt extraction, time-consuming manual data entry, and complex policy rules with numerous GL codes. These issues frequently lead to claim rejections, require manual cross-checking of multimodal documents (receipts, PDFs, itineraries), and provide no real-time validation before submission.

1.3 Sub-Task Decomposition

The proposed workflow decomposes the process into four agent nodes orchestrated through a shared state: multimodal data extraction using vision-language models, policy compliance validation using retrieval-augmented reasoning Lewis et al. [2020], and parallel compliance and fraud detection checks. A final risk-based routing node aggregates the results and conditionally directs the claim toward automated approval, correction requests, or human review. These stages cannot be merged into a single model invocation because they require different models (VLM vs. LLM), different external tools (database, currency API, email), and different reasoning contexts.

2 Agent Architecture

The system exposes a Chainlit-based Chainlit Team [2024] conversational interface supporting two user personas:

Claimant persona — Interacts with the Intake Agent via image uploads and text input. The agent guides claim preparation, flags policy violations, and iterates until all pre-submission checks pass.

Reviewer persona — Receives escalated claims with the Advisor Agent's risk summary, compliance and fraud findings, and cited policy clauses. Can approve, reject, or return the claim with comments.

2.1 System Diagram

Figure 1 presents the four-agent pipeline orchestrated by LangGraph LangChain [2024], separated into pre-submission and post-submission stages. Details of each agent's role and tools follow in Sections 2.2 and 2.3.

2.2 Agent Roles

The system is built on LangGraph, which acts as the multi-agent orchestrator, with each node hosting one or more agentic processes Anthropic [2024a] as part of the claim expense workflow:

Table 1: Agent definitions and agentic design patterns.

Agent	Pattern	Responsibility
Intake Agent	ReAct Yao et al. [2023] + Evaluator Gate	Interacts with the claimant in a conversation loop, extracts receipt data via VLM, validates against policy rules, converts currencies, and iterates until all pre-submission checks pass.
Compliance Agent	Evaluator	Audits the claim against organizational policies (budgets, spending limits, approval thresholds) and produces a pass/fail finding with cited policy clauses.
Fraud Agent	Tool call	Queries historical claims to detect fraudulent, anomalous, or duplicate receipts. Returns a fraud/legit finding per receipt.
Advisor Agent	Reflection + Routing	Synthesizes compliance and fraud findings into a risk assessment, then routes the claim to auto-approve, return, or escalate with policy citations.

2.3 Model and Tool Choices

Orchestration: LangGraph. The workflow requires conditional branching, parallel execution, iterative loops, and shared state passing between agents. LangGraph models these natively as a state machine with explicit graph nodes and edges, without requiring an additional agentic SDK.

Vision-Language Model — Candidate: GLM-4.6V GLM-V Team [2025]**; Alternative: Qwen2.5-VL-7B-Instruct.** The VLM extracts structured fields from receipt images natively, without a separate OCR pipeline. GLM-4.6V is selected for its MoE architecture and document reasoning capability. Qwen2.5-VL-7B-Instruct is the self-hostable alternative with strong OCR performance.

LLM for Reasoning Agents — Candidate: GLM-5 GLM-5 Team [2026]**; Alternative: Qwen2.5-7B-Instruct.** The LLM drives text-based reasoning across the Compliance, Fraud, and Advisor agents over policy documents and structured data. GLM-5 is selected for its agentic reasoning and 200K context window. Qwen2.5-7B-Instruct is the self-hostable alternative that eliminates API dependency.

Table 2: MCP server integrations Anthropic [2024b].

MCP Server	Used By	Purpose
RAG MCP (rag-mcp-server)	Intake, Compliance, Advisor	Retrieves relevant policy clauses from institutional knowledge base
DBHub MCP	Intake, Fraud	Provides database access via Postgres to retrieve and store structured data
Frankfurter MCP	Intake	Converts foreign currency amounts to SGD using live exchange rates
Email MCP	Advisor	Sends notification emails to claimants and reviewers

3 Multimodal Grounding

3.1 Modalities Used

The system processes three modalities across the agent pipeline:

- **Image** — Receipt images processed by a VLM to extract structured fields. Candidates: GLM-4.6V or Qwen2.5-VL-7B-Instruct.
- **Text** — Policy documents, claim descriptions, and reviewer notes validated via RAG Lewis et al. [2020]. Candidates: GLM-5 or Qwen2.5-7B-Instruct.
- **Structured Data** — Historical claims, budgets, and currency rates accessed via MCP Anthropic [2024b] tool calls. Model-agnostic.

3.2 Cross-Modal Interaction

Each agent grounds outputs from the previous stage across modalities:

- **Intake Agent** — Grounds image-extracted fields against claimant text; flags inconsistencies and returns low-confidence extractions for clarification.
- **Compliance Agent** — Grounds retrieved policy text against structured claim output; produces a pass/fail finding with cited policy clauses.
- **Fraud Agent** — Cross-references claim against historical records for duplicates, anomalies, and vendor irregularities; preserves uncertainty rather than forcing a binary outcome.
- **Advisor Agent** — Synthesizes extraction confidence, compliance findings, and fraud signals into a unified risk assessment; escalates to human review on agent disagreement.

4 Control and Failure Handling

4.1 Error Detection Mechanisms

The system detects uncertainty at each agent stage rather than treating claims as a single end-to-end prediction. The Intake Agent checks for unreadable receipts, missing fields, and low-confidence extractions before submission. Post-submission, the Compliance and Fraud Agents run in parallel, and the Advisor Agent routes the claim to auto-approve, return, or escalation.

4.2 Retry, Self-Critique, and Fallback Strategies

Each agent applies bounded retry with conservative fallback. The Intake Agent retries extraction before returning to the claimant. The Compliance Agent iterates until the outcome is clear. The Fraud Agent forwards uncertainty rather than forcing a flag on weak evidence. The Advisor Agent reflects before routing, automating only when evidence is sufficient. On tool failure, the system degrades gracefully to conservative routing or human escalation.

5 Evaluation Strategy (Preliminary)

5.1 Task-Specific Success Criteria

We evaluate the system on task-specific success metrics summarized in Table 3.

Table 3: Success criteria and measurement methods.

Metric	Target	Measurement Method
Submission time	< 3 min (vs. 15–25 min)	Average time from receipt upload to completed claim submission during a controlled user study with students submitting sample claims.
Accuracy	$> 95\%$ on structured fields	Field-level accuracy evaluation

5.2 Baselines

The system is evaluated against two baselines: (1) a single-prompt pipeline using a general-purpose model Gemini, and (2) the current manual SAP Concur workflow. Comparison targets extraction accuracy, automation rate, and workflow efficiency.

5.3 Failure Analysis

System limitations are identified using self-consistency checks, cross-modal agreement checks, verifier models, and inter-agent disagreement analysis across extraction, policy reasoning, and coordination failures.

References

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020. URL https://arxiv.org/abs/2005.11401.

Chainlit Team. Chainlit: Build conversational AI in minutes. https://github.com/Chainlit/chainlit, 2024. Open-source Python framework, Apache 2.0 License.

LangChain. LangGraph: Build stateful multi-agent applications, 2024. URL https://www.langchain.com/langgraph. Agent orchestration framework.

Anthropic. Building effective agents, 2024a. URL https://www.anthropic.com/research/building-effective-agents. Workflow and agentic pattern taxonomy.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. URL https://arxiv.org/abs/2210.03629.

GLM-V Team. GLM-4.5V and GLM-4.1V-Thinking: Towards versatile multimodal reasoning with scalable reinforcement learning, 2025. URL https://arxiv.org/abs/2507.01006.

GLM-5 Team. GLM-5: From vibe coding to agentic engineering, 2026. URL https://arxiv.org/abs/2602.15763.

Anthropic. Model context protocol, 2024b. URL https://modelcontextprotocol.io. Open protocol for tool integration with LLMs.

A System Architecture Diagram

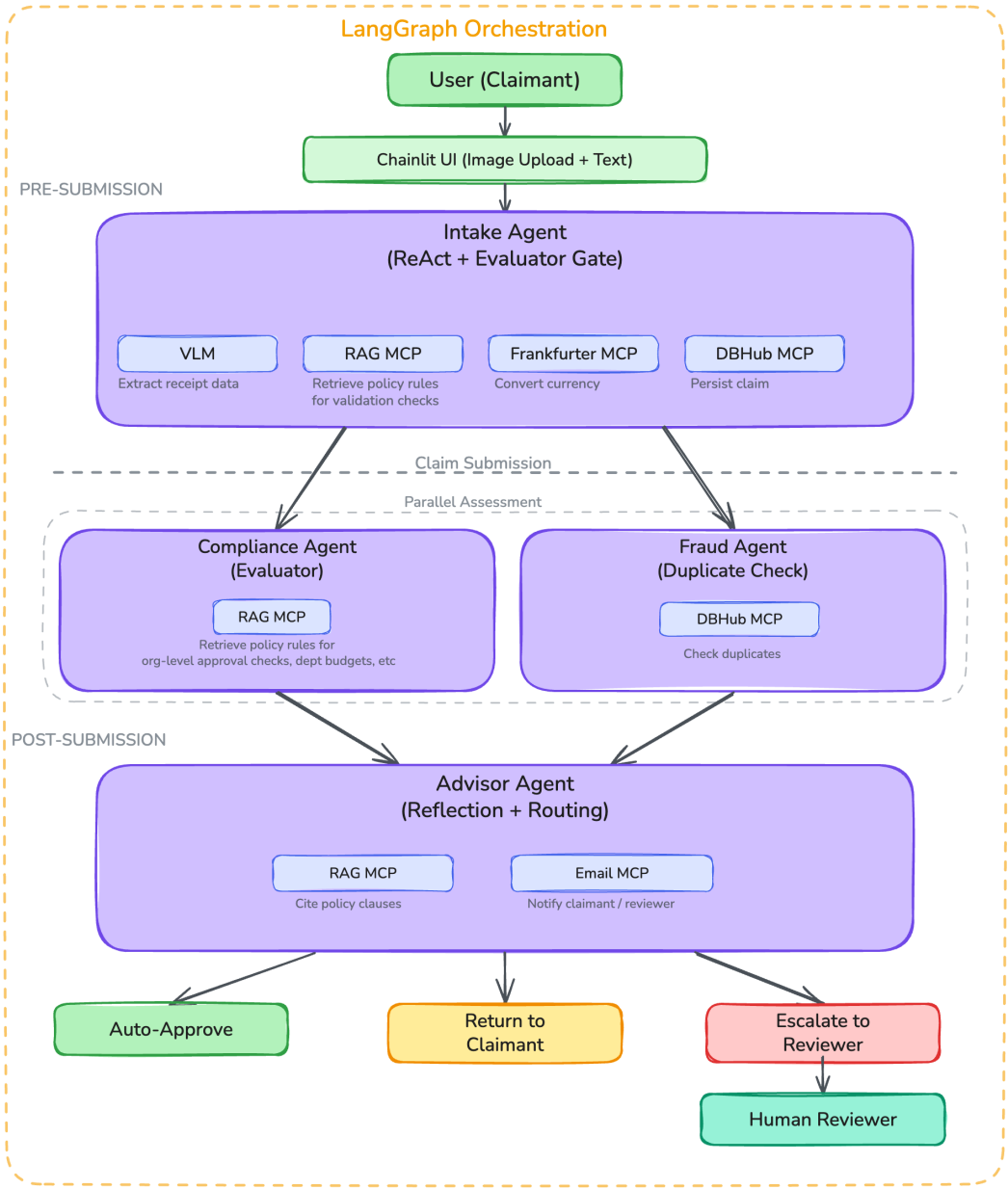

Figure 1: System diagram illustrating the stages in agentic expense workflow process, along with the respective agents, patterns and integration types.